Kiss Assembler Commands

1. Introduction

1.1 Purpose

This document is the Kiss Assembly Language command reference. Its purpose is to describe every assembler mnemonic recognised by the current Kiss toolchain, the modifier-suffix conventions that produce family-wide variant sets, and the mapping from each assembler form to an ISA command plus its Command Modifier Bit (CMB) encoding.

Where the ISA commands reference (docs/isa/kiss-isa-commands-source.md) describes the hardware-level command set from the point of view of the instruction word, this document describes the same mechanism from the point of view of the Kiss programmer writing assembly source. A single ISA command such as Add or Jmp corresponds to many assembler forms: a typed prefix selects the data type, modifier suffixes select behavioural options, a direction prefix selects PC-relative forward or backward semantics for branch families, and a condition suffix selects the flag or interrupt condition tested by the instruction. The assembler lowers each combination into the correct ISA opcode together with the CMB bit pattern that encodes the selected options.

1.2 Scope

This document covers the assembler-level view of the currently implemented Kiss command set. It is self-contained: key structural material from the ISA reference is duplicated here, so that an assembler programmer can write correct instructions without cross-referencing the full ISA document. When the two documents disagree on a matter of authoritative architectural fact, docs/isa/kiss-isa-reference.md wins.

Several command families defined in the ISA have database/tooling rows but do not yet have final simulator execution semantics. Those families — memory ordering, atomic operations, system instructions, and AI/tensor instructions — are noted under §13.4. This document distinguishes assembler/database recognition from engine behaviour: a mnemonic may assemble even when runtime semantics are currently no-op, spec-only, or not finalised.

1.3 Related documents

The assembler programmer may consult the following Kiss documents in addition to this reference:

- docs/isa/kiss-isa-reference.md — the authoritative architecture specification.
- docs/isa/kiss-isa-manual-source.md → docs/isa/kiss-isa-manual.odt — the human-readable ISA manual.
- docs/isa/kiss-isa-commands-source.md → docs/isa/kiss-isa-commands.odt — the ISA command reference (structural template for this document).
- docs/isa/kiss-isa-operand-notation.md — compact operand-role notation reference.
- docs/assembler-language/kiss-assembler-grammar.md — the authoritative assembler grammar specification.

2. Conventions

2.1 Mnemonic casing

Kiss assembler mnemonics use mixed-case canonical spellings. Canonical forms include Move, SetVal, SetTxt, Test, Compare, and ChkBnds. The deprecated short forms Mov, Tst, Cmp, and CHKBNDS are no longer accepted by the assembler. Note that SetImm is the underlying **ISA** command — it is not a mnemonic at the assembler level; the programmer writes SetVal or SetTxt. The assembler is case-sensitive in its reference output; tooling that historically used upper-case or truncated names should be updated to the canonical forms. The deprecated forms are catalogued in §13.3.

2.2 Dotted datatype prefix

Where a command is typed, the assembler form begins with a dotted datatype prefix:

- B. — Binary
- I. — Integer
- D. — Decimal

For example, B.Add, I.Sub, and D.Mul are the three datatype forms of the untyped ISA command Add, Sub, and Mul. The prefix is lowered by the assembler into CMB 1–2 of the emitted instruction (01 Binary, 10 Integer, 11 Decimal).

The Cvt command uses a double-dotted source-destination prefix — B.I.Cvt converts Binary to Integer, D.B.Cvt converts Decimal to Binary — with the source type encoded in CMB 1–2 and the destination type encoded in CMB 7–8 (see §12).

2.3 Modifier suffixes

Most typed command families accept an ordered sequence of modifier suffixes appended to the base mnemonic. The standard suffix order is:

```
<Type>.<Base><Direction><Carry|Borrow><Saturation><SetFlags>
```

Not every family uses every slot. The full set of modifier suffixes accepted by one or more families is:

- L / R — direction (Left / Right) for rotate, shift, copy, move, load, store, and I/O families.
- Cry — carry-in for Add (multi-precision arithmetic chain). Not used by rotate / shift families.
- Brw — borrow-in for Sub (multi-precision arithmetic chain).
- Sat — saturating behaviour on overflow or underflow.
- Set — update the processor status flags after the operation.

Example: I.AddCrySatSet — Integer, Add, with carry-in, Saturating, Set flags.

Not every combination is valid. Family-specific restrictions are given in each section. Two global rules apply:

- The Apply-to families (AddTo, SubTo, MulTo, DivTo) permit only the base form and Sat. Carry/borrow and Set are not valid for these commands (§8.5).
- Incr / Decr do not accept Sat, Cry, or Brw; they accept only the base form and Set (§8.6).

2.4 Direction prefix for Jump and Call

Jump and Call families use a **prefix** to select PC-relative forward or backward addressing, not a suffix. The prefix is placed before the base mnemonic (including any condition suffix):

- Jmp / Call — absolute address.
- FJmp / FCall — PC-relative forward.
- BJmp / BCall — PC-relative backward.

The direction prefix combines with the condition suffix: FJmpZ, BJmpLE, FCallNE, BCallGT are all valid. Old suffix-form direction mnemonics (JmpF, JmpR, CallF, CallR) are not accepted; see §13.3.

2.5 Condition suffixes

Jump, Call, and If commands share a set of 30 condition suffixes. Each suffix has a fixed meaning regardless of the family it is attached to, and each corresponds to a flag-based test performed at execution time. The complete table is given in §14.2.

The If family accepts one additional condition not available to Jump or Call: IfIRQ tests whether an interrupt request is pending. IRQ is not part of the shared 30-suffix set because it names a hardware state rather than an arithmetic result.

2.6 Operand notation

This document uses compact operand-role notation when describing operand layouts:

- R — register ID.
- sR — source register; read, not modified.
- dR — destination register; written.
- sdR — source-and-destination register; read, then written (in-place).
- iR — immediate or register; CMB-3 selects.
- asR — source-address register; holds an address, memory at that address is read.
- adR — destination-address register; holds an address, memory at that address is written.
- o prefix — optional operand (e.g., osR3, odR4, osdR2).

The numeric suffix is the operand slot, 1 through 5. Example: dR5 means operand 5 is a destination register.

2.7 Indexing and widths

Kiss uses 1-based indexing everywhere. Section numbers, operand slots, bit positions, register IDs, stack entries, and array indices all start at 1. The value 0 is reserved as the invalid or unused sentinel — for example, an operand slot filled with register ID 0 means “not used”.

The Kiss byte is **10 bits**, not 8. A Kiss instruction is **100 bits** wide. Assembler byte counts (Load, Stor, Read, Write) count Kiss bytes.

2.8 Status-flag naming

Status-flag suffixes used in condition names use the following short forms:

- **Parity:** Evn (even), Odd (odd).
- **Sign:** Pos (positive), Neg (negative), NPos, NNeg.
- **Compare:** EQ, NE, GT, LT, GE, LE, Abv, AbvE, Blw, BlwE.
- **Arithmetic exceptions:** Ovr (overflow), Und (underflow), Wrp (modular wrap), Cry (carry), Brw (borrow), Inf (decimal ±infinity), AnyCarry (decimal carry/borrow digit pending).
- **Negations:** prefix N — e.g., NOvr, NWrp, NCry, NBrw.

Ovr / Und / Wrp form a four-case matrix: Ovr/Und fire on any over/under-capacity outcome, while Wrp fires only when the outcome was a modular wrap (Add/Sub crossing capacity). Mul precision overflow and Div with a nonzero remainder set Ovr/Und without Wrp — destination is a closest-

representable approximation, not a meaningful modular result. Cry is set only by Add and Brw only by Sub; both are direction-agnostic, indicating the result didn't fit.

2.9 Literal type tags

Immediate literals in assembler source may carry an explicit type tag:

- #B — unsigned Binary literal.
- #I — signed Integer literal (Kiss ones complement; bit 1 = MSB = 1 for positive).
- #D — Decimal literal (packed decimal).
- #T — Text literal (packed Kiss character codes).

A literal without a tag is resolved from context — the expected operand type or the typed prefix of the enclosing instruction determines the encoding. The assembler packs the literal into the correct bit pattern before emission; the CPU sees only opaque bits.

Text literals are currently written with double-quote delimiters: "abc". Guillemet delimiters («abc») are an open design item and are not yet accepted by the current lexer.

3. Common instruction structure

This section duplicates the structural material an assembler programmer needs to emit valid Kiss instructions. For the full architectural narrative, see docs/isa/kiss-isa-reference.md and §3 of docs/isa/kiss-isa-commands-source.md.

3.1 Instruction word layout

Every Kiss instruction is 100 bits wide and is laid out as follows:

Field	Bits	Width
Conditional Execution Bits (CEB)	1–5	5
Command	6–17	12
Command Modifier Bits (CMB)	18–25	8
Operand 1	26–40	15
Operand 2	41–55	15
Operand 3	56–70	15
Operand 4	71–85	15
Operand 5	86–100	15

The CEB field is identified bit by bit as CEB-1 through CEB-5. The CMB field is identified bit by bit as CMB-1 through CMB-8. CMB-1 is instruction bit 18; CMB-8 is instruction bit 25.

The SetImm family uses a special-form layout in which Operand 1 carries the destination register and Operands 2–5 together carry a 50-bit immediate payload, right-justified in the 60-bit span

(bits 41–50 are unused padding). See §13.1 for the full SetImm encoding.

3.2 Conditional Execution Bits

CEB-1 is the conditional-execution enable: 0 means the instruction is unconditional, 1 means it belongs to a conditional block. CEB-2 through CEB-5 encode the active nesting level as a one-hot mask. The valid CEB patterns are:

CEB	Meaning
`00000`	Unconditional / top-level
`11000`	First nested `If` level
`10100`	Second nested `If` level
`10010`	Third nested `If` level
`10001`	Fourth nested `If` level

Four nesting levels are supported. An instruction executes only when its CEB matches the thread's Main CEB register; otherwise it is suppressed as a NOP. If* pushes Main to Shadow, increments Depth, and activates the next level on a true test; EndIf restores Shadow to Main and decrements Depth; ClrCF resets all three CEB fields — Main, Shadow, and Depth — to their initial values (Main = Shadow = 00000, Depth = 0).

Jump commands carry a CEB clear count in CMB 6–8 so that a branch out of one or more nested If* blocks unwinds the correct number of levels without executing matching EndIfs. Call commands save the full CEB state to the shadow call stack and enter the subroutine at 00000.

3.3 Command field

The 12-bit Command field selects the base ISA operation. For typed commands (Add, Sub, Mul, Div, And, Or, …) the base opcode is untyped and the data type is carried in the CMB. For condition-bearing families (Jump, Call, If) each condition has its own opcode, so that the condition is fast-decoded from the Command field alone.

3.4 Command Modifier Bits

The eight CMB bits modify the behaviour of the base command. Two assignments are nearly universal:

CMB	Common meaning
CMB 1–2	Data type: `00` untyped / bit-processing, `01` Binary, `10` Integer, `11` Decimal.
CMB-3	Immediate-value indicator: `0` operand is a register ID, `1` operand is an immediate value.
CMB-6	Set Status Flags: `0` leave flags, `1` update flags.
CMB 7–8	Immediate-value data type (when CMB-3 = `1`): same encoding as CMB 1–2.

The meaning of CMB-4, CMB-5, and (in some families) CMB 7–8 varies. Each family's section below gives the exact layout.

Any CMB bit not defined for a given command must be 0. Nonzero reserved bits may trap or produce implementation-defined behaviour.

3.5 Operand slots

Operands 1 through 5 each carry a 15-bit field — large enough to hold a register ID, a small immediate, or an address offset. Semantics vary by family:

- **Destination-last.** The primary destination is operand 5. Fanout destinations (where supported) fill backward from operand 4. Sources fill forward from operand 1.
- **Register ID 0.** 0 in a register-ID slot means “not used”. Optional operands set to 0 are treated as absent.
- **SetImm exception.** In the SetImm family, the destination is in operand 1 and operands 2–5 carry the immediate payload.
- **Push / Pull exception.** Push and Pull do not use destination-last — operand 1 is the stack selector, and operands 2–5 are processed in order (R2 first, then R3, R4, R5) so that stack order is preserved.

3.6 Sign convention for Integer

Kiss Integer uses ones-complement representation. Bit 1 of an Integer register is the most significant bit (MSB) and is the sign bit: 1 means the value is **positive**, 0 means the value is **negative**. This convention is the opposite of the usual twos-complement sign bit and is critical for correct hand-assembled Integer literals.

3.7 CMB quick reference

The compact table below summarises the most common bit meanings. Individual families may override any of these; consult the relevant family section for the exact layout.

CMB bit	Absolute	Common meaning
CMB 1–2	18–19	Data type (`00` untyped, `01` Binary, `10` Integer, `11` Decimal)
CMB-3	20	Immediate-value indicator
CMB-4	21	Family-specific (direction, saturation, width-immediate, …)
CMB-5	22	Family-specific (carry / borrow, backward, DivRem selector, …)
CMB-6	23	Set Status Flags
CMB 7–8	24–25	Immediate data type, destination type, or unused

4. Control and conditional-execution commands

4.1 Overview

Control and conditional-execution commands manage the execution of the current thread rather

than the movement of data. The family splits into two groups:

- **Execution control** — NOP, Brk, Retrn, ClrCF.
- **Conditional block** — the If* family (27 variants including IfIRQ) and EndIf.

Every command in this family uses no CMB bits (all eight CMB bits are 0) and most use no operands. The condition, where present, is encoded in the Command field itself.

4.2 Execution control

Assembler	ISA Command	CMB	Operands	Description
`NOP`	`NOP`	`00000000`	—	No operation; pipeline placeholder
`Brk`	`Brk`	`00000000`	—	Breakpoint / debugger trap
`Retrn`	`Retrn`	`00000000`	—	Return from subroutine; restore saved state from shadow call stack
`ClrCF`	`ClrCF`	`00000000`	—	Reset CEB state (Main = Shadow = `00000`, Depth = 0)

Retrn restores R1–R4, the Status Flag register (SF), CEB state, and the special-register set from the protected shadow call stack. R5 is not restored — it is the parameter / return-value bank. ClrCF is the explicit recovery mechanism when normal If* / EndIf unwinding cannot be relied on.

4.3 Conditional block (If / EndIf)

Each If* variant evaluates a named condition. When the condition is true the command pushes Main CEB to Shadow, increments Depth, and activates the next nested CEB value in Main. When the condition is false, no new level is activated and the block is skipped. EndIf restores Shadow → Main and decrements Depth, closing exactly one nesting level.

Assembler	ISA Command	Condition
`IfIRQ`	`IfIRQ`	Interrupt-request pending
`IfZ`	`IfZ`	Zero flag set
`IfNZ`	`IfNZ`	Zero flag clear
`IfBlw`	`IfBlw`	Below (unsigned)
`IfBlwE`	`IfBlwE`	Below or equal (unsigned)
`IfLT`	`IfLT`	Less than (signed)
`IfLE`	`IfLE`	Less than or equal (signed)
`IfEQ`	`IfEQ`	Equal
`IfNE`	`IfNE`	Not equal
`IfGE`	`IfGE`	Greater than or equal (signed)
`IfGT`	`IfGT`	Greater than (signed)
`IfAbvE`	`IfAbvE`	Above or equal (unsigned)
`IfAbv`	`IfAbv`	Above (unsigned)
`IfPos`	`IfPos`	Positive

Assembler	ISA Command	Condition
`IfNPos`	`IfNPos`	Not positive
`IfNeg`	`IfNeg`	Negative
`IfNNeg`	`IfNNeg`	Not negative
`IfEvn`	`IfEvn`	Even
`IfNEvn`	`IfNEvn`	Not even
`IfOdd`	`IfOdd`	Odd
`IfNOdd`	`IfNOdd`	Not odd
`IfOvr`	`IfOvr`	Overflow flag set
`IfNOvr`	`IfNOvr`	Overflow flag clear
`IfUnd`	`IfUnd`	Underflow flag set
`IfNUnd`	`IfNUnd`	Underflow flag clear
`IfBrw`	`IfBrw`	Borrow flag set
`IfNBrw`	`IfNBrw`	Borrow flag clear
`IfCry`	`IfCry`	Carry flag set
`IfNCry`	`IfNCry`	Carry flag clear
`IfTHi`	`IfTHi`	High-end truncation flag set
`IfNTHi`	`IfNTHi`	High-end truncation flag clear
`IfTLo`	`IfTLo`	Low-end truncation flag set
`IfNTLo`	`IfNTLo`	Low-end truncation flag clear
`EndIf`	`EndIf`	Closes one active If level

All 34 commands (33 If* variants plus EndIf) have all eight CMB bits set to 0 and take no operands. The condition is fully encoded in the Command opcode.

The assembler keyword Else is not an ISA command. It is lowered by the assembler into an EndIf closing the current If block followed by a new complementary If* opening a block with the opposite condition. The emitted instruction stream contains only standard If* / EndIf pairs. See §13.2 for the detailed lowering.

5. Jump commands

5.1 Overview

Jump commands transfer execution to a target instruction location without saving a return address. The Jump family provides three base assembler forms — Jmp, FJmp, and BJmp — and 30 conditional base commands, one per shared condition suffix. Each conditional variant is a distinct ISA opcode.

- Jmp — absolute target address.
- FJmp — PC-relative forward (target = PC + Opr1).

- BJmp — PC-relative backward (target = PC − Opr1).

Every conditional Jump also supports the three direction forms via the prefix mechanism: JmpZ / FJmpZ / BJmpZ, JmpLE / FJmpLE / BJmpLE, and so on.

The target operand is iR1: an immediate address or a register containing the address, selected by CMB-3.

5.2 CMB usage

- **CMB 1–2** — Immediate-value data type (when CMB-3 = 1).
- **CMB-3** — Operand-1 source kind: 0 register, 1 immediate.
- **CMB-4** — PC-relative mode: 0 absolute, 1 PC-relative.
- **CMB-5** — Backward selector (meaningful only when CMB-4 = 1): 0 forward, 1 backward.
- **CMB 6–8** — CEB clear count. Three bits encoding how many active conditional-execution levels to unwind if the jump is taken.

The three base forms correspond to:

- Jmp — CMB-4 = 0, CMB-5 = 0.
- FJmp — CMB-4 = 1, CMB-5 = 0.
- BJmp — CMB-4 = 1, CMB-5 = 1.

5.3 Jump mnemonics

Assembler	ISA Command	Description
`Jmp`	`Jmp`	Unconditional jump
`JmpZ`	`JmpZ`	Jump if zero
`JmpNZ`	`JmpNZ`	Jump if not zero
`JmpBlw`	`JmpBlw`	Jump if below (unsigned)
`JmpBlwE`	`JmpBlwE`	Jump if below or equal
`JmpLT`	`JmpLT`	Jump if less than (signed)
`JmpLE`	`JmpLE`	Jump if less than or equal
`JmpEQ`	`JmpEQ`	Jump if equal
`JmpNE`	`JmpNE`	Jump if not equal
`JmpGE`	`JmpGE`	Jump if greater than or equal
`JmpGT`	`JmpGT`	Jump if greater than
`JmpAbvE`	`JmpAbvE`	Jump if above or equal
`JmpAbv`	`JmpAbv`	Jump if above
`JmpPos`	`JmpPos`	Jump if positive

Assembler	ISA Command	Description
`JmpNPos`	`JmpNPos`	Jump if not positive
`JmpNeg`	`JmpNeg`	Jump if negative
`JmpNNeg`	`JmpNNeg`	Jump if not negative
`JmpEvn`	`JmpEvn`	Jump if even
`JmpOdd`	`JmpOdd`	Jump if odd
`JmpOvr`	`JmpOvr`	Jump if overflow
`JmpNOvr`	`JmpNOvr`	Jump if not overflow
`JmpUnd`	`JmpUnd`	Jump if underflow
`JmpNUnd`	`JmpNUnd`	Jump if not underflow
`JmpBrw`	`JmpBrw`	Jump if borrow
`JmpNBrw`	`JmpNBrw`	Jump if not borrow
`JmpCry`	`JmpCry`	Jump if carry
`JmpNCry`	`JmpNCry`	Jump if not carry
`JmpTHi`	`JmpTHi`	Jump if high-end truncation occurred
`JmpNTHi`	`JmpNTHi`	Jump if no high-end truncation
`JmpTLo`	`JmpTLo`	Jump if low-end truncation occurred
`JmpNTLo`	`JmpNTLo`	Jump if no low-end truncation

Every one of the 31 base mnemonics above also accepts the F and B direction prefixes — for example FJmpZ, BJmpEQ, FJmpNCry, BJmpOvr, FJmpTHi. That produces 31 × 3 = 93 distinct assembler mnemonics in the Jump family, all sharing the same operand and CMB layout.

5.4 Operand usage

- **Operand 1 (iR1):** the jump target. Immediate or register per CMB-3.
- **Operands 2–5:** unused, must be 0.

5.5 Notes

CMB 6–8 carry the CEB clear count so that a jump out of nested If* blocks unwinds the correct number of levels without passing through the matching EndIfs. The assembler computes this count from the source structure when lowering the instruction.

6. Call commands

6.1 Overview

Call commands transfer execution to a target for subroutine invocation. Like Jumps, the Call family provides three base forms — Call, FCall, BCall — and 30 condition-bearing variants, each a distinct ISA opcode. Every conditional Call supports the three direction forms (e.g., CallNE / FCallNE / BCallNE).

When a Call executes, the processor saves protected call-frame state onto the shadow call stack (R1–R4, the Status Flag register, CEB state, and the special-register set) and then enters the called routine with CEB = 00000. R5 is not saved — it is the parameter / return-value bank. The matching Retrn restores the saved state.

6.2 CMB usage

Call uses the same first five CMB bits as Jump (§5.2). The difference is CMB 6–8:

- **CMB 6–8** — Unused for Call. Must be 0.

A Call does not need a CEB clear count because the protected save/restore mechanism captures the entire CEB state on entry and restores it on Retrn. Branches that merely exit some nested If* blocks without calling a subroutine use Jump, not Call.

6.3 Call mnemonics

Assembler	ISA Command	Description
`Call`	`Call`	Unconditional call
`CallZ`	`CallZ`	Call if zero
`CallNZ`	`CallNZ`	Call if not zero
`CallBlw`	`CallBlw`	Call if below (unsigned)
`CallBlwE`	`CallBlwE`	Call if below or equal
`CallLT`	`CallLT`	Call if less than (signed)
`CallLE`	`CallLE`	Call if less than or equal
`CallEQ`	`CallEQ`	Call if equal
`CallNE`	`CallNE`	Call if not equal
`CallGE`	`CallGE`	Call if greater than or equal
`CallGT`	`CallGT`	Call if greater than
`CallAbvE`	`CallAbvE`	Call if above or equal
`CallAbv`	`CallAbv`	Call if above
`CallPos`	`CallPos`	Call if positive
`CallNPos`	`CallNPos`	Call if not positive
`CallNeg`	`CallNeg`	Call if negative
`CallNNeg`	`CallNNeg`	Call if not negative
`CallEvn`	`CallEvn`	Call if even
`CallOdd`	`CallOdd`	Call if odd
`CallOvr`	`CallOvr`	Call if overflow
`CallNOvr`	`CallNOvr`	Call if not overflow
`CallUnd`	`CallUnd`	Call if underflow
`CallNUnd`	`CallNUnd`	Call if not underflow
`CallBrw`	`CallBrw`	Call if borrow

Assembler	ISA Command	Description
`CallNBrw`	`CallNBrw`	Call if not borrow
`CallCry`	`CallCry`	Call if carry
`CallNCry`	`CallNCry`	Call if not carry
`CallTHi`	`CallTHi`	Call if high-end truncation occurred
`CallNTHi`	`CallNTHi`	Call if no high-end truncation
`CallTLo`	`CallTLo`	Call if low-end truncation occurred
`CallNTLo`	`CallNTLo`	Call if no low-end truncation

As with Jump, every base mnemonic also accepts the F and B direction prefixes — FCallZ, BCallEQ, FCallNCry, FCallTHi, and so on — for a total of 31 × 3 = 93 distinct Call mnemonics.

6.4 Operand usage

- **Operand 1 (iR1):** the call target. Immediate or register per CMB-3.
- **Operands 2–5:** unused, must be 0.

6.5 Notes

The shadow call stack is a hardware-protected resource distinct from the general data stacks (see §7.3). Its depth is implementation-defined but large enough to accommodate deeply nested calls. Retrn always pops exactly one entry and restores the matching saved state. The return location is not an explicit operand — it is recovered from the saved state.

7. Data movement commands

7.1 Overview

Data movement commands transfer data between registers, memory, the stack, and hardware I/O ports without transforming the values being moved. The family splits into five sub-families:

- **Load and Store** — memory ↔ register.
- **Push and Pull** — register ↔ stack.
- **Copy and Move** — register ↔ register.
- **Read and Write** — register ↔ I/O port.
- **Register utilities** — ClrReg, Swap.

Each sub-family has its own CMB layout and operand pattern. The direction-explicit L / R mnemonic suffix recurs across Load / Store, Copy / Move, and Read / Write, and is always encoded in CMB-4.

7.2 Load and Store (LoadR / LoadL / StorR / StorL)

Load reads a span of memory into a destination register; Store writes a span of a source register to memory. Both are type-agnostic (CMB 1–2 = 00): they move raw bytes, with the byte count supplied as operand 1.

The assembler provides three address-computation modes per direction:

- LoadL / LoadR, StorL / StorR — effective address is the base address directly.
- LoadLOfs / LoadROfs, StorLOfs / StorROfs — effective address is base + offset.
- LoadLOfsMul / LoadROfsMul, StorLOfsMul / StorROfsMul — effective address is base + (offset × multiplier).

7.2.1 CMB usage

- **CMB 1–2** — Unused. Must be 00.
- **CMB-3** — Immediate indicator for operand 1 (byte count): 0 register, 1 immediate.
- **CMB-4** — Direction: 0 right-aligned (*R mnemonic), 1 left-aligned (*L mnemonic).
- **CMB-5** — Offset enable: 0 base only, 1 base + offset.
- **CMB-6** — Multiplier enable: 0 no multiplier, 1 multiplier present (valid only when CMB-5 = 1).
- **CMB 7–8** — Unused. Must be 0.

CMB-5 = 0 with CMB-6 = 1 (multiplier without offset) is invalid.

7.2.2 Mnemonics

Assembler	ISA Command	CMB-4	CMB-5	CMB-6	Description
`LoadR`	`Load`	`0`	`0`	`0`	Load Opr1 bytes from mem[R2], right-aligned
`LoadL`	`Load`	`1`	`0`	`0`	Load, left-aligned
`LoadROfs`	`Load`	`0`	`1`	`0`	Load from mem[R2 + R3], right-aligned
`LoadLOfs`	`Load`	`1`	`1`	`0`	Load from mem[R2 + R3], left-aligned
`LoadROfsMul`	`Load`	`0`	`1`	`1`	Load from mem[R2 + R3 × R4], right-aligned
`LoadLOfsMul`	`Load`	`1`	`1`	`1`	Load from mem[R2 + R3 × R4], left-aligned
`StorR`	`Stor`	`0`	`0`	`0`	Store Opr1 bytes of R2 to mem[R3], right-aligned
`StorL`	`Stor`	`1`	`0`	`0`	Store, left-aligned
`StorROfs`	`Stor`	`0`	`1`	`0`	Store to mem[R3 + R4], right-aligned
`StorLOfs`	`Stor`	`1`	`1`	`0`	Store to mem[R3 + R4], left-aligned
`StorROfsMul`	`Stor`	`0`	`1`	`1`	Store to mem[R3 + R4 × R5], right-aligned
`StorLOfsMul`	`Stor`	`1`	`1`	`1`	Store to mem[R3 + R4 × R5], left-aligned

7.2.3 Operand usage

Operand 1 is always a byte count (iR1), immediate or register per CMB-3.

Load forms:

- LoadL | LoadR iR1 asR2 dR5 — byte count, base address, destination.
- LoadLOfs | LoadROfs iR1 asR2 sR3 dR5 — add register offset.
- LoadLOfsMul | LoadROfsMul iR1 asR2 sR3 sR4 dR5 — add register offset × multiplier.

Store forms:

- StorL | StorR iR1 sR2 adR3 — byte count, source, base address.
- StorLOfs | StorROfs iR1 sR2 adR3 sR4 — add register offset.
- StorLOfsMul | StorROfsMul iR1 sR2 adR3 sR4 sR5 — add register offset × multiplier.

Offset and multiplier operands are currently register-only; immediate offsets and multipliers are not supported by the present encoding.

7.3 Push and Pull

Push writes register contents onto a data stack; Pull reads register contents back from a data stack. Kiss supports up to 10 independent data stacks per thread, numbered 1 through 10. Stack 1 is the default when no explicit stack is given.

Stack entries are variable-sized: when data is pushed, a size field is appended after the data, so that Pull can determine how much to unload without requiring the caller to know the size at the pull site.

7.3.1 CMB usage

- **CMB 1–2** — Unused. Must be 00.
- **CMB-3** — Immediate indicator for operand 1 (stack selector).
- **CMB-4, CMB 5–6** — Unused. Must be 0.
- **CMB 7–8** — Immediate data type (when CMB-3 = 1).

7.3.2 Mnemonics

Assembler	ISA Command	Description
`Push`	`Push`	Push register(s) onto stack
`Pull`	`Pull`	Pull register(s) from stack

7.3.3 Operand usage

- **Operand 1 (iR1):** stack selector (1–10).
- **Operand 2 (sR2 for Push, dR2 for Pull):** first register to transfer.
- **Operands 3–5 (osR / odR):** optional additional registers, processed in order.

A single Push or Pull instruction moves up to four registers. Push / Pull do not follow the destination-last convention — registers are processed in the operand order R2, R3, R4, R5 so that stack ordering is preserved.

7.4 Copy and Move

Copy duplicates a source register value into one or more destination registers. Move is identical except that the source register is zeroed after all destinations have been written. Both are typed commands — the datatype prefix controls how the value is extended, truncated, or sign-adjusted when the source and destination widths differ.

7.4.1 CMB usage

- **CMB 1–2** — Data type (01 Binary, 10 Integer, 11 Decimal).
- **CMB-3** — Unused. Must be 0.
- **CMB-4** — Direction: 0 right-aligned (*R), 1 left-aligned (*L).
- **CMB 5–8** — Unused. Must be 0.

7.4.2 Mnemonics

Assembler	ISA Command	CMB 1–2	CMB-4	Description
B.CopyL	Copy	01	1	Copy Binary, left-aligned
B.CopyR	Copy	01	0	Copy Binary, right-aligned
I.CopyL	Copy	10	1	Copy Integer, left-aligned
I.CopyR	Copy	10	0	Copy Integer, right-aligned
D.CopyL	Copy	11	1	Copy Decimal, left-aligned
D.CopyR	Copy	11	0	Copy Decimal, right-aligned
B.MoveL	Move	01	1	Move Binary, left-aligned
B.MoveR	Move	01	0	Move Binary, right-aligned
I.MoveL	Move	10	1	Move Integer, left-aligned
I.MoveR	Move	10	0	Move Integer, right-aligned
D.MoveL	Move	11	1	Move Decimal, left-aligned
D.MoveR	Move	11	0	Move Decimal, right-aligned

7.4.3 Operand usage

- **Operand 1 (sR1):** source register.
- **Operands 2–4 (odR2, odR3, odR4):** optional fanout destinations.
- **Operand 5 (dR5):** primary destination.

A single Copy or Move can fan out the source value to up to four destinations. For Move, the source register is zeroed only after all destinations have received their copies.

7.4.4 Integer sign-aware resize

When CMB 1–2 = 10 (Integer) and the source and destination register widths differ, Copy and Move automatically sign-extend or truncate with sign preservation. Kiss Integer uses ones complement with bit 1 as the MSB: 1 means positive, 0 means negative. Expanding pads the extra high-order positions with NOT(sign bit); shrinking keeps the low-order value bits and forces the sign bit to match the original. Binary, Decimal, and untyped (CMB 1–2 = 00) copies are raw bit moves with no sign awareness. Each fanout destination is independently resized.

7.5 Read and Write (I/O)

Read and Write transfer bytes between registers and hardware I/O ports. Both are type-agnostic (CMB 1–2 = 00), consistent with Load and Stor. The byte count is operand 1.

7.5.1 CMB usage

- **CMB 1–2** — Unused. Must be 00.
- **CMB-3** — Immediate indicator for byte count.
- **CMB-4** — Direction: 0 right-aligned (*R), 1 left-aligned (*L).
- **CMB 5–8** — Unused / reserved. Must be 0.

7.5.2 Mnemonics

Assembler	ISA Command	CMB-4	Description
`ReadR`	`Read`	`0`	Read bytes from port, right-aligned
`ReadL`	`Read`	`1`	Read bytes from port, left-aligned
`WriteR`	`Write`	`0`	Write bytes to port, right-aligned
`WriteL`	`Write`	`1`	Write bytes to port, left-aligned

The direction suffix is mandatory — there is no unsuffixed Read or Write form.

7.5.3 Operand usage

- **Read:** Read iR1 sR2 dR5 — byte count, port selector, destination.
- **Write:** Write iR1 sR2 sR3 — byte count, port selector, source.

7.6 Register utilities

ClrReg zeros one or more registers; Swap exchanges the contents of two registers. Both use no CMB modifiers.

Assembler	ISA Command	CMB	Description
`ClrReg`	`ClrReg`	`00000000`	Clear register(s) to zero
`Swap`	`Swap`	`00000000`	Exchange two registers

ClrReg takes a primary destination dR5 and up to four optional destinations (odR4, odR3, odR2, odR1) filling backward from operand 4. Each listed register is zeroed.

Swap takes two operands — sdR1 and sdR5 — and exchanges their values. Operands 2, 3, and 4 are unused.

7.7 SetVal and SetTxt (immediate values)

SetVal and SetTxt load an immediate value directly into a register. Both assembler mnemonics lower to the same underlying ISA command, SetImm, which uses a special-form layout: the destination register is in Operand 1, and the 50-bit immediate payload is packed into Operands 2–5. The assembler chooses between the two mnemonics based on payload kind:

- SetVal — numeric payload (CMB-4 = 0).
- SetTxt — text payload (CMB-4 = 1).

See §13.1 for the full encoding and assembler behaviour.

Assembler	ISA Command	CMB-4	Description
`SetVal`	`SetImm`	`0`	Load immediate numeric value into destination register
`SetTxt`	`SetImm`	`1`	Load immediate text value into destination register

8. Arithmetic commands

8.1 Overview

The arithmetic family performs computational operations on register values. Every arithmetic command is typed — the assembler form carries a B., I., or D. prefix — and every family in this section shares a common CMB layout referred to as Pattern A. This section covers:

- **Ordinary arithmetic:** Add, Sub, Mul, Div.
- **Fused multiply-add:** MulAdd.
- **Divide with remainder:** DivRem.
- **Apply-to:** AddTo, SubTo, MulTo, DivTo.
- **Increment and Decrement:** Incr, Decr.

8.2 Pattern A CMB usage

Pattern A is the CMB layout shared by the ordinary arithmetic commands. Related families (MulAdd, DivRem, Apply-to, Incr / Decr) apply the same layout with small modifications described in each sub-section.

- **CMB 1–2** — Data type (01 Binary, 10 Integer, 11 Decimal).
- **CMB-3** — Immediate indicator.
- **CMB-4** — Saturation: 0 wrap, 1 saturate.
- **CMB-5** — Carry / borrow: command-specific (see §8.3).
- **CMB-6** — Set Status Flags.
- **CMB 7–8** — Reserved or immediate data type depending on family.

8.3 Ordinary arithmetic (Add, Sub, Mul, Div)

The ordinary arithmetic family follows Pattern A. CMB-5 has a command-specific meaning:

- **Add** — 1 includes the current carry value (AddCry).
- **Sub** — 1 includes the current borrow value (SubBrw).
- **Mul, Div** — unused; must be 0.

CMB 7–8 are reserved for ordinary arithmetic and must be 0.

8.3.1 Mnemonics

Assembler	ISA Command	CMB 1–2	Modifiers	Description
`B.Add`	`Add`	`01`	—	Binary addition
`I.Add`	`Add`	`10`	—	Integer addition
`D.Add`	`Add`	`11`	—	Decimal addition
`B.AddSet`	`Add`	`01`	CMB-6=1	Binary add, set flags
`I.AddSet`	`Add`	`10`	CMB-6=1	Integer add, set flags
`D.AddSet`	`Add`	`11`	CMB-6=1	Decimal add, set flags
`B.AddCry`	`Add`	`01`	CMB-5=1	Binary add with carry-in
`I.AddCry`	`Add`	`10`	CMB-5=1	Integer add with carry-in
`D.AddCry`	`Add`	`11`	CMB-5=1	Decimal add with carry-in
`B.AddSat`	`Add`	`01`	CMB-4=1	Binary add, saturating
`I.AddSat`	`Add`	`10`	CMB-4=1	Integer add, saturating
`D.AddSat`	`Add`	`11`	CMB-4=1	Decimal add, saturating
`B.AddCrySatSet`	`Add`	`01`	CMB-4=1, 5=1, 6=1	Binary add with carry, saturating, set flags

Assembler	ISA Command	CMB 1–2	Modifiers	Description
`B.Sub`	`Sub`	`01`	—	Binary subtraction
`I.Sub`	`Sub`	`10`	—	Integer subtraction
`D.Sub`	`Sub`	`11`	—	Decimal subtraction
`B.SubBrw`	`Sub`	`01`	CMB-5=1	Binary subtract with borrow-in
`I.SubBrw`	`Sub`	`10`	CMB-5=1	Integer subtract with borrow-in
`D.SubBrw`	`Sub`	`11`	CMB-5=1	Decimal subtract with borrow-in
`B.Mul`	`Mul`	`01`	—	Binary multiplication
`I.Mul`	`Mul`	`10`	—	Integer multiplication
`D.Mul`	`Mul`	`11`	—	Decimal multiplication
`D.MulCry`	`Mul`	`11`	CMB-5=1	Decimal multiply with carry-in (long-mul chain)
`B.Div`	`Div`	`01`	—	Binary division
`I.Div`	`Div`	`10`	—	Integer division
`D.Div`	`Div`	`11`	—	Decimal division
`D.DivBrw`	`Div`	`11`	CMB-5=1	Decimal divide with borrow-in (long-div chain)

Every ordinary arithmetic command accepts the full Pattern A suffix set. Suffixes combine in the canonical order Cry / Brw, then Sat, then Set: I.SubBrwSatSet, D.MulSatSet, B.AddCrySet, and so on.

Decimal Mul / Div carry/borrow chain. Cry and Brw are valid on Decimal Mul and Div only — see §8.3.4 for the chain semantics. Bin/Int Mul/Div remain Cry/Brw-invalid (multi-precision binary algorithms don't fit a single-bit chain). MulAdd and DivRem are also Cry/Brw-invalid for the reasons noted in §8.3 and the carve-out doc.

8.3.2 Operand usage

- **Operand 1 (sR1):** first source.
- **Operand 2 (sR2):** second source.
- **Operands 3–4 (osR3, osR4):** optional additional sources (chained — R5 = R1 op R2 op R3 op R4).
- **Operand 5 (dR5):** destination.

Optional sources chain into the same operator. The destination is always operand 5.

8.3.3 Add / Sub carry-borrow chain math (Bin / Int / Dec)

AddCry reads a carry-in digit from the SCRY register and computes R1 + R2 + carry_in → R5. The result's "didn't-fit" digit is written back to SCRY for the next step's chain. SubBrw is symmetric using SBRW. For Bin/Int the chain is single-bit; for Dec it's a single 5-bit digit (the high overflow / new remainder). See decisions.md 2026-03-28 for the original spec.

8.3.4 Decimal Mul / Div chain math (D.MulCry, D.DivBrw)

Per the 2026-05-08 carve-out (docs/historical/decimal-update-plan.md §Item 3, supersedes the 2026-03-28 blanket "Cry/Brw not valid on Mul/Div" decision for Decimal only):

- D.MulCry**:** computes (R1 × R2) + carry_in_at_LSD → R5. The carry-in digit is read from SCRY and **added at the LSD of the product** (matching the D.AddCry pattern). The highest digit lost beyond the destination's capacity is written back to SCRY for chain continuation. Intended for column-by-column long multiplication: R1 is the multi-digit multiplicand, R2 is one column digit of the multi-digit multiplier, SCRY is the previous column's overflow digit.
- D.DivBrw**:** computes the long-division step (borrow_in × 10^DigitsR1 + R1) ÷ R2 → R5. The borrow-in digit (previous column's remainder) is read from SBRW and **prepended to R1 as the new high-order digit** — different from AddCry / SubBrw / MulCry, which inject at the LSD. This matches standard long-division where each column's remainder feeds into the next column's effective dividend at the high end. The new remainder's lowest digit is written back to SBRW.
- MulAdd **and** DivRem **are excluded** from the carve-out. MulAdd's two-stage operation has no clean single-Cry chain semantic; DivRem already exposes the remainder via Operand 4, so a separate Brw chain bit would be redundant.
- **Bin / Int Mul / Div remain Cry/Brw-invalid.** Multi-precision binary multiplication and long division use shift-and-add and bit-level long-division algorithms that don't reduce to a single-bit chain.

For multi-digit divisors (D.DivBrw) or multi-digit second operands (D.MulCry), the math is computed correctly but the single-digit chain register only captures one digit of overflow / remainder — useful information may be lost. These operations are designed for the single-digit-column case typical of Decimal long arithmetic.

8.4 Fused multiply-add (MulAdd) and divide-with-remainder (DivRem)

8.4.1 MulAdd

MulAdd computes (R1 × R2) + R3 → R5 as a fused operation, with the intermediate product not subject to overflow before the addition.

- **CMB 1–2** — Data type.
- **CMB-3** — Immediate indicator.
- **CMB-4** — Saturation applies to the final result.
- **CMB-5** — Unused. Must be 0. Carry/borrow modifiers are not valid for MulAdd per the 2026-03-28 decision.

- **CMB-6** — Set Status Flags.

Assembler	ISA Command	CMB 1–2	Description
`B.MulAdd`	`MulAdd`	`01`	Binary (R1 × R2) + R3 → R5
`I.MulAdd`	`MulAdd`	`10`	Integer fused multiply-add
`D.MulAdd`	`MulAdd`	`11`	Decimal fused multiply-add

Supported suffixes: Sat, Set, and the combination SatSet. Cry and Brw are not valid.

Operand pattern: <Type>.MulAdd sR1 sR2 sR3 dR5 — multiplicand, multiplier, addend, destination. Operand 4 is unused.

8.4.2 DivRem

DivRem computes R1 ÷ R2, writing the quotient to R5 and the remainder to R4.

- **CMB 1–2** — Data type.
- **CMB-3** — Immediate indicator.
- **CMB-4** — Saturation.
- **CMB-5** — Always 1; this bit distinguishes DivRem from Div.
- **CMB-6** — Set Status Flags.

Saturation is valid for DivRem. Carry/borrow modifiers are not valid (CMB-5 is not available — it is consumed by the DivRem discriminator).

Assembler	ISA Command	CMB 1–2	Description
`B.DivRem`	`DivRem`	`01`	Binary divide → quotient + remainder
`I.DivRem`	`DivRem`	`10`	Integer divide → quotient + remainder
`D.DivRem`	`DivRem`	`11`	Decimal divide → quotient + remainder

Supported suffixes: Sat, Set, and SatSet.

Operand pattern: <Type>.DivRem sR1 sR2 dR4 dR5 — dividend, divisor, remainder destination, quotient destination. Operand 3 is unused.

Divide-by-zero behavior.

- B.DivRem / I.DivRem with divisor 0: faults (ARITH_OVERFLOW / div-by-zero) unless Sat is active. With Sat, the quotient clamps by direction (max-positive on positive-direction overflow, 0 on negative-direction underflow) and the remainder destination receives 0 (Binary and Integer have no representable infinity).
- D.DivRem with divisor 0: **never faults.** Both quotient and remainder receive ±∞, with sign matching dividend_sign XNOR divisor_sign — Kiss Decimal has signed zeros (+0, −0), so the divisor's sign matters. The Inf flag is set when Set is active. With Sat, both destinations instead clamp to all-9s of matching sign. The "remainder = ∞ when divisor = 0" semantic

matches HLL §4.1 (Dec Mod 0 = ∞); infinity is a first-class Decimal value, so it propagates loudly through subsequent arithmetic rather than masking the divide-by-zero condition.

8.5 Apply-to (AddTo, SubTo, MulTo, DivTo)

The Kiss-50 Apply-to family applies a single value (operand 1) in place to one target register. The target is read, the operation is performed with operand 1, and the result is written back to the same target.

Modifier restriction. Per the 2026-03-28 decision, the Apply-to commands accept only the base form and Sat. The Set, Cry, and Brw suffixes are **not valid** for these commands. Apply-to does not set flags or consume carry/borrow; programmers needing those behaviours should use the ordinary Add / Sub / Mul / Div commands.

8.5.1 Kiss-50 CMB usage

- **CMB 1–2** — Data type.
- **CMB-3** — Unused; must be 0 (Set not valid).
- **CMB-4** — Saturation.
- **CMB-5** — Immediate value selector for operand 1: 0 = Opr1 is a value register, 1 = Opr1 is a compact immediate value.

The Apply-to family permits a compact immediate value in operand 1. Set, Cry, and Brw remain invalid; CMB-5 is used only for the Opr1 immediate/register distinction, not carry/borrow.

8.5.2 Mnemonics

Assembler	ISA Command	CMB 1–2	Description
`B.AddTo`	`AddTo`	`01`	Add value to Binary target(s) in place
`I.AddTo`	`AddTo`	`10`	Add value to Integer target(s) in place
`D.AddTo`	`AddTo`	`11`	Add value to Decimal target(s) in place
`B.SubTo`	`SubTo`	`01`	Subtract value from Binary target(s) in place
`I.SubTo`	`SubTo`	`10`	Subtract value from Integer target(s) in place
`D.SubTo`	`SubTo`	`11`	Subtract value from Decimal target(s) in place
`B.MulTo`	`MulTo`	`01`	Multiply Binary target(s) by value in place
`I.MulTo`	`MulTo`	`10`	Multiply Integer target(s) by value in place
`D.MulTo`	`MulTo`	`11`	Multiply Decimal target(s) by value in place
`B.DivTo`	`DivTo`	`01`	Divide Binary target(s) by value in place
`I.DivTo`	`DivTo`	`10`	Divide Integer target(s) by value in place
`D.DivTo`	`DivTo`	`11`	Divide Decimal target(s) by value in place

Only Sat is a valid suffix: B.AddToSat, I.SubToSat, D.MulToSat, and so on.

8.5.3 Operand usage

<Type>.AddTo iR1 sdR2 — value in operand 1 and target in operand 2 for Kiss-50. CMB-5 distinguishes compact immediate value (1) from value register (0). The target is read, combined with the value in operand 1, and written back.

8.6 Increment and Decrement (Incr, Decr)

Incr and Decr are amount-driven in-place updates. The amount is an explicit operand — they are not ±1 shortcuts. Kiss-50 uses a single target operand.

Modifier restriction. Incr and Decr accept only the base form and Set. Sat, Cry, and Brw are **not valid**.

8.6.1 Kiss-50 CMB usage

- **CMB 1–2** — Data type.
- **CMB-3** — Set Status Flags.
- **CMB-4** — Unused (must be 0; saturation not valid).
- **CMB-5** — Immediate amount selector for operand 1: 0 = Opr1 is an amount register, 1 = Opr1 is a compact immediate amount.

Sat, Cry, and Brw remain invalid for Incr/Decr. CMB-5 is used only for the Opr1 immediate/register distinction, not carry/borrow.

8.6.2 Mnemonics

Assembler	ISA Command	CMB 1–2	Description
`B.Incr`	`Incr`	`01`	Add an explicit amount to one or more Binary registers
`I.Incr`	`Incr`	`10`	Add an explicit amount to one or more Integer registers
`D.Incr`	`Incr`	`11`	Add an explicit amount to one or more Decimal registers
`B.Decr`	`Decr`	`01`	Subtract an explicit amount from one or more Binary registers
`I.Decr`	`Decr`	`10`	Subtract an explicit amount from one or more Integer registers
`D.Decr`	`Decr`	`11`	Subtract an explicit amount from one or more Decimal registers

Only Set is a valid suffix: B.IncrSet, I.DecrSet, and so on.

8.6.3 Operand usage

<Type>.Incr iR1 sdR2 — amount in operand 1 and target in operand 2 for Kiss-50. CMB-5 distinguishes compact immediate amount (1) from amount register (0). The target is read, adjusted by the amount, and written back.

<Type>.Decr iR1 sdR2 — same Kiss-50 operand pattern, subtracting the amount. CMB-5 distinguishes compact immediate amount (1) from amount register (0).

9. Rotate and shift commands

9.1 Overview

Three ISA commands form the rotate-and-shift family:

ISA Command	Operation	Type coverage
`Rot`	Rotate (bits/digits shifted out of one end re-enter the opposite end)	B / I / D
`Shift`	Logical shift (vacated positions filled with zero or sign-matched zero)	B / I / D
`SShift`	Signed (arithmetic) shift (sign bit preserved)	I only

Rot and Shift accept all three data types. SShift applies to Integer only — Binary has no sign concept, and Decimal's signed-digit encoding (sign embedded in every digit) makes a "preserve the sign bit at MSB" semantic ill-defined. All three commands share an identical CMB layout and accept direction and flag-setting modifiers.

9.2 CMB usage

- **CMB 1–2** — Data type.
- **CMB-3** — Immediate indicator for the shift / rotate count (operand 1).
- **CMB-4** — Direction: 0 right, 1 left.
- **CMB-5** — Unused. Must be 0.
- **CMB-6** — Set Status Flags.
- **CMB 7–8** — Immediate data type.

9.3 Naming pattern

<Type>.<Family><Direction>[Set]

- **Type** — B, I, or D (Integer only for SShift).
- **Family** — Rot, Shift, or SShift.
- **Direction** — L or R.
- **Set** — set status flags.

The leading S in SShift is part of the family name (Signed Shift), not a Set-flag suffix. A trailing Set is the only indicator for flag-setting.

9.4 Mnemonics

Assembler	ISA Command	CMB 1–2	CMB-4	Description
B.RotL	Rot	01	1	Binary rotate left
B.RotR	Rot	01	0	Binary rotate right
I.RotL	Rot	10	1	Integer rotate left
I.RotR	Rot	10	0	Integer rotate right
D.RotL	Rot	11	1	Decimal rotate left (digit-level)
D.RotR	Rot	11	0	Decimal rotate right (digit-level)
B.ShiftL	Shift	01	1	Binary logical shift left
B.ShiftR	Shift	01	0	Binary logical shift right
I.ShiftL	Shift	10	1	Integer logical shift left
I.ShiftR	Shift	10	0	Integer logical shift right
D.ShiftL	Shift	11	1	Decimal logical shift left (digit-level)
D.ShiftR	Shift	11	0	Decimal logical shift right (digit-level)
I.SShiftL	SShift	10	1	Integer signed shift left
I.SShiftR	SShift	10	0	Integer signed shift right

Every mnemonic also accepts the Set suffix to update status flags. Examples: B.RotRSet, I.ShiftLSet, D.RotLSet, I.SShiftRSet.

9.5 Operand usage

- **Operand 1 (iR1):** shift / rotate count (immediate or register). For Binary and Integer the count is in bits; for Decimal it is in digits (5-bit packed digits).
- **Operands 2–4 (osdR2, osdR3, osdR4):** optional additional targets, filled backward.
- **Operand 5 (sdR5):** primary target.

The count applies uniformly to all listed targets. A single instruction rotates or shifts up to four registers by the same count.

9.6 Behaviour

- **Rot** — circular rotation.
  - **Binary / Integer:** bit-level rotation. Bits shifted out of one end re-enter at the opposite end.
  - **Decimal:** digit-level rotation. 5-bit packed digits shifted out of one end re-enter at the opposite end. Null-coded digits are skipped — they remain in their physical positions while the non-Null digits rotate around them.
- **Shift** — logical shift. Bits/digits shifted off the end are lost.

    - **Binary / Integer:** bit-level shift. Vacated positions filled with 0.
    - **Decimal:** digit-level shift. Vacated digit positions filled with the sign-matched zero digit — Pos_0 if the value's sign (taken from the first non-Null digit) is positive, Neg_0 if negative. Null-coded digits and the decimal-point marker are skipped — they remain in their physical positions while the non-Null digits shift around them. Lost non-Null digits are silent — the Shift / Rot family does not set Ovr / Und / Wrp on bit-loss (Spec C, 2026-05-08).

- **SShift** (Integer only) — arithmetic shift. The sign bit (high-order bit, bit 1 per Kiss convention) is preserved at the MSB across the shift; only the magnitude bits move. **Vacated magnitude positions fill with the inverse of the sign bit** — 0 for positive Int (sign bit 1), 1 for negative Int (sign bit 0). This applies to both directions: SShiftR vacates high-order magnitude positions, SShiftL vacates low-order magnitude positions, and both fill with NOT(sign bit). The rule is the inverse of x86's "fill with sign bit" SAR semantic — Kiss reverses it because Kiss's sign convention is reversed (sign-bit-1 = positive). Lost magnitude bits are silent — the Shift / Rot family does not set Ovr / Und / Wrp on bit-loss (Spec C, 2026-05-08).

10. Logic and bit-processing commands

10.1 Overview

This section covers two related groups:

- **Logic families** — And, AndN, NAnd, Or, OrN, NOr, XOr, NXOr, Not, Neg.
- **Bit-processing** — BitFlip, BitOn, BitOff, BitTstOn, BitTstOff, BitRead, BitWrite, BitClear, BitCnt, *BitFind*, BitRev, ByteRev.

Logic-family commands are typed and share the simplest CMB layout in the ISA: data type plus optional Set Status Flags. Bit-processing commands use CMB 1–2 = 00 (untyped / bit-processing) and carry their own family-specific CMB meanings.

10.2 Logic-family CMB usage

- **CMB 1–2** — Data type (01 Binary, 10 Integer, 11 Decimal).
- **CMB 3–5** — Unused. Must be 0.
- **CMB-6** — Set Status Flags.
- **CMB 7–8** — Unused. Must be 0.

This layout is shared by every logic-family command including Not and Neg.

10.3 Ordinary logic

The eight ordinary logic commands perform standard two-input bitwise operations.

Assembler	ISA Command	CMB 1–2	Description
B.And	And	01	Binary AND
I.And	And	10	Integer AND
D.And	And	11	Decimal AND
B.AndN	AndN	01	Binary AND-NOT (A AND ¬B)
I.AndN	AndN	10	Integer AND-NOT
D.AndN	AndN	11	Decimal AND-NOT
B.NAnd	NAnd	01	Binary NAND
I.NAnd	NAnd	10	Integer NAND
D.NAnd	NAnd	11	Decimal NAND
B.Or	Or	01	Binary OR
I.Or	Or	10	Integer OR
D.Or	Or	11	Decimal OR
B.OrN	OrN	01	Binary OR-NOT (A OR ¬B)
I.OrN	OrN	10	Integer OR-NOT
D.OrN	OrN	11	Decimal OR-NOT
B.NOr	NOr	01	Binary NOR
I.NOr	NOr	10	Integer NOR
D.NOr	NOr	11	Decimal NOR
B.XOr	XOr	01	Binary XOR
I.XOr	XOr	10	Integer XOR
D.XOr	XOr	11	Decimal XOR
B.NXOr	NXOr	01	Binary exclusive NOR
I.NXOr	NXOr	10	Integer exclusive NOR
D.NXOr	NXOr	11	Decimal exclusive NOR

All 24 forms accept the Set suffix (CMB-6 = 1): B.AndSet, I.OrSet, D.XOrSet, and so on.

Operand pattern: <Type>.<Op> sR1 sR2 dR5 — two sources, one destination. Operands 3 and 4 are unused.

10.4 Not and Neg

10.4.1 Not

Not performs bitwise inversion (ones complement) in place. Unlike the ordinary logic commands, Not operates in-place and supports fanout: a single instruction can invert up to five registers.

Assembler	ISA Command	CMB 1–2	Description
`B.Not`	`Not`	`01`	Binary bitwise NOT
`I.Not`	`Not`	`10`	Integer bitwise NOT
`D.Not`	`Not`	`11`	Decimal bitwise NOT

With Set (CMB-6 = 1), Not is restricted to a single target in operand 5, so that the flag state reflects one deterministic result. Without Set, operands 1–5 may all be in-place targets.

10.4.2 Neg

Neg performs arithmetic negation (sign inversion) on a single register in place. Arithmetic negation is meaningful only for signed types; **there is no Binary Neg**.

Assembler	ISA Command	CMB 1–2	Description
`I.Neg`	`Neg`	`10`	Integer negation
`D.Neg`	`Neg`	`11`	Decimal negation
`I.NegSet`	`Neg`	`10`	Integer negation, set flags
`D.NegSet`	`Neg`	`11`	Decimal negation, set flags

An encoding with CMB 1–2 = 01 (Binary) and the Neg opcode is reserved and must not be emitted.

Operand pattern: <Type>.Neg sdR5 — single in-place target. No fanout.

10.5 Bit-processing overview

Bit-processing commands manipulate individual bits or bit-fields within a register. They use the Bit* prefix (no dot) and are inherently binary — there are no typed forms. CMB 1–2 is always 00 for this family, which is otherwise an unused datatype code.

ByteRev uses a Byte prefix rather than Bit because it operates on Kiss bytes (10-bit units), not individual bits, but it shares the CMB 1–2 = 00 convention.

10.6 BitFlip

BitFlip is semantically identical to Not but exists as a distinct ISA command to provide an explicit bit-manipulation name within the bit-processing family. Its operand pattern and Set behaviour follow Not (§10.4.1).

Assembler	ISA Command	CMB 1–2	Description
`BitFlip`	`BitFlip`	`00`	Invert all bits in one or more registers (in place)

Operand pattern: BitFlip [osdR1] [osdR2] [osdR3] [osdR4] sdR5.

10.7 Single-bit set and test

BitOn and BitOff are assembler forms for ISA BitSet; BitTstOn and BitTstOff are forms for ISA BitTst. Bit positions are 1-based (bit 1 is the least significant bit).

10.7.1 CMB usage

- **CMB 1–2** — 00 (bit-processing).
- **CMB-3** — Immediate indicator for bit position (operand 1): 0 register, 1 immediate.
- **CMB-4** — On/Off selector: 1 target value is 1 (On), 0 target value is 0 (Off).
- **CMB-5** — Unused.
- **CMB-6** — Set Status Flags — required 1 for BitTst (these commands exist to set the Z flag); 0 for BitSet.
- **CMB 7–8** — Unused.

10.7.2 Mnemonics

Assembler	ISA Command	CMB-4	CMB-6	Description
`BitOn`	`BitSet`	`1`	`0`	Set a single bit to `1`
`BitOff`	`BitSet`	`0`	`0`	Clear a single bit to `0`
`BitTstOn`	`BitTst`	`1`	`1`	Test whether a bit is `1`; CMB-6 Set is encoded, Z reflects test result
`BitTstOff`	`BitTst`	`0`	`1`	Test whether a bit is `0`; set Z if so

Operand pattern for BitOn, BitOff: iR1 sdR5 — bit position, target register (in place). Single target; no fanout.

Operand pattern for BitTstOn, BitTstOff: iR1 sR5 — bit position, source register. Sets the Z flag based on the bit value; subsequent JumpIf Z or IfZ branches on the test result.

10.8 Bit-field operations (BitRead, BitWrite, BitClear)

These three commands operate on a contiguous range of bits specified by a start position and a width.

10.8.1 CMB usage

- **CMB 1–2** — 00.
- **CMB-3** — Immediate indicator for operand 1 (start position).
- **CMB-4** — Immediate indicator for operand 2 (width).
- **CMB-5** — Reserved.
- **CMB-6** — Set Status Flags.
- **CMB 7–8** — Reserved.

10.8.2 Mnemonics

Assembler	ISA Command	Description
`BitRead`	`BitRead`	Read a bit-field from a source register; result low-aligned in destination
`BitWrite`	`BitWrite`	Insert a low-aligned bit-field into a target register at a given position
`BitClear`	`BitClear`	Zero a bit-field at a given position

Operand patterns:

- BitRead iR1 iR2 sR3 dR4 — start position, width, source, destination.
- BitWrite iR1 iR2 sR3 sdR4 — start position, width, source (low-aligned value), target (in place).
- BitClear iR1 iR2 sdR3 — start position, width, target (in place).

Read and Write accurately describe the semantics: no shifting is implied beyond the low-alignment of the transferred bits in BitRead's destination and BitWrite's source.

10.9 Bit counting

These six commands count set or clear bits in a source register. All share the same operand pattern.

10.9.1 CMB usage (BitCnt / BitCntTot)

- **CMB 1–2** — 00.
- **CMB-3** — For BitCnt: 1 leading, 0 trailing. Unused for BitCntTot.
- **CMB-4** — On / Off selector: 1 count 1-bits, 0 count 0-bits.
- **CMB-6** — Set Status Flags.

10.9.2 Mnemonics

Assembler	ISA Command	CMB-3	CMB-4	Description
`BitCntTotOn`	`BitCntTot`	—	`1`	Count total `1` bits (population count)
`BitCntTotOff`	`BitCntTot`	—	`0`	Count total `0` bits
`BitCntLeadOn`	`BitCnt`	`1`	`1`	Count leading `1` bits from the MSB
`BitCntLeadOff`	`BitCnt`	`1`	`0`	Count leading `0` bits from the MSB
`BitCntTrailOn`	`BitCnt`	`0`	`1`	Count trailing `1` bits from the LSB
`BitCntTrailOff`	`BitCnt`	`0`	`0`	Count trailing `0` bits from the LSB

Operand pattern: sR1 dR5 — source, destination. The count is written to dR5.

10.10 Bit finding (BitFindFstOn, BitFindLstOn)

Assembler	ISA Command	CMB-3	CMB-4	Description
`BitFindFstOn`	`BitFind`	`1`	`1`	Find the first `1`-bit position from the left (MSB side)
`BitFindLstOn`	`BitFind`	`0`	`1`	Find the last `1`-bit position from the right (LSB side)

Result is a 1-based position; 0 means “not found” (consistent with the Kiss indexing directive). BitFindFstOn and BitFindLstOn encode CMB-6 Set, so when the result is 0, the Z flag is set and JumpIf Z or IfZ can branch on “not found”.

Operand pattern: sR1 dR5.

10.11 Bit and byte reversal

Assembler	ISA Command	Description
`BitRev`	`BitRev`	Reverse bit order in a register
`ByteRev`	`ByteRev`	Reverse byte (10-bit unit) order in a register

CMB 1–2 = 00 for both; CMB-6 optionally 1 for Set flags. ByteRev uses a Byte prefix because it operates on Kiss bytes, not individual bits.

Operand pattern: sR1 dR5.

11. Test, compare, and bounds commands

11.1 Overview

Three ISA commands make up this family: **Test**, **Compare**, and **ChkBnds**. All are typed. Their sole purpose is to evaluate a condition and update the status flags — they do not produce a result in a destination register.

11.2 CMB usage

- **CMB 1–2** — Data type.
- **CMB 3–5** — Unused. Must be 0.
- **CMB-6** — Set Status Flags. **Required to be** 1 for every instance of this family — the commands exist to set flags.
- **CMB 7–8** — Unused. Must be 0.

11.3 Mnemonics

Assembler	ISA Command	CMB 1–2	Description
`B.Test`	`Test`	`01`	Test a Binary register against zero

Assembler	ISA Command	CMB 1–2	Description
`I.Test`	`Test`	`10`	Test an Integer register against zero
`D.Test`	`Test`	`11`	Test a Decimal register against zero
`B.Compare`	`Compare`	`01`	Compare two Binary values
`I.Compare`	`Compare`	`10`	Compare two Integer values
`D.Compare`	`Compare`	`11`	Compare two Decimal values
`B.ChkBnds`	`ChkBnds`	`01`	Check a Binary value against inclusive low/high bounds
`I.ChkBnds`	`ChkBnds`	`10`	Check an Integer value against inclusive bounds
`D.ChkBnds`	`ChkBnds`	`11`	Check a Decimal value against inclusive bounds

11.4 Operand usage

- **Test:** <Type>.Test sR1 — test R1 against zero; sets the Result-property group (Z / Pos / Neg / Evn / Odd) and the Compare group's signed quartet (EQ / GT / GE / LT / LE relative to zero), without disturbing the Arithmetic group.
- **Compare:** <Type>.Compare sR1 sR2 — compare R1 vs R2; sets the Compare group's signed quartet (EQ / GT / GE / LT / LE), without disturbing the Arithmetic or Result-property groups.
- **ChkBnds:** <Type>.ChkBnds sR1 sR2 sR3 — test whether R2 ≤ R1 ≤ R3 inclusively; sets bounds flags (EQ + signed quartet).

The magnitude directional quartet (Abv/AbvE/Blw/BlwE, bits 18–21) is set only by CmpMag, which is **spec-defined but not yet implemented**. Test, Compare, and ChkBnds leave those four bits untouched — they remain 0 until CmpMag lands. See docs/isa/kiss-isa-commands-source.md §11.8 for the full CmpMag spec.

All operand registers are read-only. No register is modified by these commands.

12. Conversion commands

12.1 Overview

A single ISA command, Cvt, handles all supported data-type conversion paths. The assembler uses a double-dotted source-destination prefix to produce a distinct assembler form for each conversion.

12.2 CMB usage

Cvt uses a unique CMB layout with two datatype fields:

- **CMB 1–2** — Source data type (01 Binary, 10 Integer, 11 Decimal).
- **CMB 3–6** — Unused. Must be 0.

- **CMB 7–8** — Destination data type (same encoding as CMB 1–2).

No other Kiss command uses both CMB 1–2 and CMB 7–8 as datatype fields.

12.3 Mnemonics

Assembler	ISA Command	CMB 1–2 (source)	CMB 7–8 (dest)	Description
`B.I.Cvt`	`Cvt`	`01`	`10`	Binary → Integer
`B.D.Cvt`	`Cvt`	`01`	`11`	Binary → Decimal
`I.B.Cvt`	`Cvt`	`10`	`01`	Integer → Binary
`I.D.Cvt`	`Cvt`	`10`	`11`	Integer → Decimal
`D.B.Cvt`	`Cvt`	`11`	`01`	Decimal → Binary
`D.I.Cvt`	`Cvt`	`11`	`10`	Decimal → Integer

Kiss Decimal is packed decimal; the Decimal conversion forms above handle packing and unpacking directly. No separate packed-decimal conversion commands exist.

12.4 Operand usage

<Src>.<Dst>.Cvt sR1 dR5 — source register (read, not modified), destination register. Operands 2–4 are unused. No fanout.

Same-type conversions (e.g., B.B.Cvt) are encodable but semantically a no-op copy. Whether the processor treats them as valid or raises a fault is implementation-defined.

13. Special cases

13.1 SetVal and SetTxt

The SetVal and SetTxt assembler mnemonics both compile to the single ISA command SetImm. SetImm is a special-form command whose instruction layout differs from the standard destination-last convention:

- **Operand 1** carries the **destination register** (not operand 5).
- **Operands 2–5** together carry a **50-bit immediate payload**, right-justified in the 60-bit span. Bits 41–50 of operand 2 are unused padding and must be 0.

This layout was redesigned on 2026-04-09; the previous encoding (destination in Opr 5, 45-bit payload) is deprecated — see §13.3.

13.1.1 CMB usage

- **CMB 1–2** — Data type. 00 = untyped, 01 = Binary, 10 = Integer, 11 = Decimal. The CPU uses CMB 1–2 to apply type-aware extension when the destination register is wider than 50 bits — see §13.1.4 below. (Prior to the 2026-05-08 / 05-09 type-awareness fix, CMB 1–2

was treated as informational only; the CPU now reinterprets the 50-bit payload per the CMB 1–2 datatype during the store.)

- **CMB-3** — Immediate-form indicator; always 1 for SetImm.
- **CMB-4** — Numeric / text selector: 0 SetVal (numeric payload), 1 SetTxt (text payload).
- **CMB 5–8** — Unused. Must be 0.

CMB-4 also selects the payload alignment within the destination register:

- **SetVal (CMB-4 = 0):** numeric payload, right-aligned at the destination's LSD end (with Integer sign extension and Decimal digit re-alignment per §13.1.4).
- **SetTxt (CMB-4 = 1):** text payload, left-aligned in the destination — the first characters land at the high end.

13.1.2 Operand layout

- **Operand 1 (dR1):** destination register.
- **Operands 2–5:** the 50-bit payload, right-justified across the 60-bit span (bits 41–50 of the instruction word are padding).

The 50-bit payload width is a hard limit on the amount of data a single SetVal or SetTxt instruction can embed.

13.1.3 Assembler lowering

Given source such as SetVal R1WA #B:12345 or SetTxt R2WA "abc", the assembler:

1. Reads the literal and its explicit type tag (#B, #I, #D, #T) or infers the type from context.
2. Converts the literal to the appropriate 50-bit pattern (unsigned binary, ones-complement integer, packed 5-bit decimal digits, or packed Kiss character codes).
3. Places the destination register identifier into operand 1.
4. Packs the 50-bit payload across operands 2–5 (right-justified; high 10 bits of operand 2 zero-padded).
5. Sets CMB-3 = 1, CMB-4 = 0 for SetVal or 1 for SetTxt, and CMB 1–2 to the literal's data type (00 untyped, 01 Binary, 10 Integer, 11 Decimal). Per §13.1.5, the engine reads CMB 1–2 to apply type-aware extension when the destination is wider than 50 bits.

13.1.4 Mnemonics

Assembler	ISA Command	CMB-4	Description
`SetVal`	`SetImm`	`0`	Load numeric immediate into destination register
`SetTxt`	`SetImm`	`1`	Load text immediate into destination register

13.1.5 Type-aware width extension (numeric SetVal)

When the destination register is wider than 50 bits, the CPU extends the 50-bit payload to the full destination width using rules selected by CMB 1–2. Without this extension, a 50-bit Integer immediate written raw to a 100-bit register would put the sign bit at bit 49 instead of bit 99 where Integer arithmetic looks for it; Decimal digits would land in the middle of the register instead of right-aligned at the LSD end.

- **CMB 1–2 =** 10 **(Integer):** Sign-extends per Kiss reversed-sign convention. The 50-bit Int's sign bit (bit 49 of the payload) is placed at the destination's MSB; bits 49 through (LenBits−2) are filled with NOT(sign bit) per the Kiss extension rule (0 for positive, 1 for negative); magnitude bits 0–48 are preserved.
- **CMB 1–2 =** 11 **(Decimal):** Right-aligns the encoded digits at the destination's LSD end. The encoder packs N digits MSB-first at the top of the 50-bit field with Null padding in the low slots; the engine counts trailing Null digits and shifts the immediate right by (trailing_nulls × 5) bits so the actual N digits land at the destination's LSD-most N positions, with leading positions Null. The decimal-point marker (DecPt) is treated as a non-Null digit and travels with the data. *Single-digit example:* SetVal R, #D:5 on a one-byte (10-bit) destination produces a byte with 00000 in the high half (bits 1–5, the MSD slot) and the digit code 10101 (+5) in the low half (bits 6–10, the LSD slot).
- **CMB 1–2 =** 01 **(Binary) or** 00 **(untyped):** No extension. The 50-bit payload is written to the destination's low 50 bits; high bits remain zero. Binary has no sign concept; untyped is opaque.
- **CMB-4 =** 1 **(SetTxt):** Left-aligned regardless of CMB 1–2. The high bits of the 50-bit immediate hold the first characters; subsequent characters fill toward the destination's LSD end up to the 50-bit limit.

For destination registers ≤ 50 bits, no extension is applied. Integer values narrower than 50 bits truncate the magnitude to fit while preserving the sign bit at the destination's MSB.

This behavior was introduced in commits 8726b7a (Integer, 2026-05-08) and 905ba83 (Decimal, 2026-05-09); see docs/decisions.md 2026-05-09 entry.

13.2 Else keyword

The assembler keyword Else is not an ISA command. It is a structural convenience provided by the assembler that lowers into standard If* / EndIf pairs plus CEB manipulation. The emitted instruction stream contains only standard If* and EndIf instructions; no "Else" opcode exists at the ISA level.

The lowering of Else under an If* block is:

1. Emit an EndIf closing the current If block.

2. Emit a new If* opening a block with the complementary condition.

Because Else is purely a compile-time construct, it has no CMB encoding and no operands in the emitted stream.

13.3 Deprecated forms

The following assembler forms are historical and are no longer accepted by the current Kiss toolchain:

Deprecated form	Replaced by	Notes
`Mov`	`Move`	Short form superseded by the canonical mixed-case name.
`Cmp`	`Compare`	Short form superseded.
`Tst`	`Test`	Short form superseded.
`SETIMM`	— (use `SetVal` or `SetTxt`)	There has never been a `SetImm` assembler mnemonic; `SetImm` is the underlying ISA command that the assembler forms `SetVal` / `SetTxt` lower to. The upper-case spelling is superseded and was never a valid assembler form.
`JmpF`, `JmpR`	`FJmp`, `BJmp`	Direction is now a **prefix** (F/B), not a suffix. Similarly for all conditional forms (`JmpZF` → `FJmpZ`, `JmpEQR` → `BJmpEQ`, etc.).
`CallF`, `CallR`	`FCall`, `BCall`	Same prefix-direction change as Jump.
SetImm encoding with destination in Opr 5 and 45-bit payload	SetImm encoding with destination in Opr 1 and 50-bit payload across Opr 2–5	Superseded by the 2026-04-09 redesign. The earlier layout is no longer emitted or accepted.

Source files using any of the above forms should be updated to the canonical mnemonics and the current SetImm encoding.

13.4 Scope notes

Several command families defined in the Kiss ISA database (Kiss10.db) are intentionally out of scope for this document. They are listed here so that programmers encountering their names in design documents know where they stand:

Family	ISA commands	Current status
Memory ordering	`Fence`, `FenceI`	Recognised by the database/tooling. The current single-core in-order simulator handles these as no-ops until memory-ordering behaviour graduates beyond no-op.
Atomic operations	`Amo*` family plus reservation forms	Recognised by the database/tooling where rows exist. Runtime semantics are spec-only until multi-core / SMT memory semantics are finalised.
System instructions	`Sys.*` family (16 commands)	Recognised by the database/tooling. Runtime semantics are spec-only until the kernel/user mode mechanism is finalised.
AI / Tensor	`TileLoad`, `TileStor`, `MtxMulAdd`, `ActReLU`,	Recognised by the database/tooling where rows

Family	ISA commands	Current status
	`ActSig`, `ActSMax`, `DotPrd`, `RowSum`, `ColSum`, `Quant`, `DeQuant`	exist. Runtime semantics are spec-only until tensor-unit architecture is finalised.

For the above families, this document records assembler-visible spelling and encoding where the rows exist; detailed runtime behaviour should be treated as pending until simulator semantics are finalised.

14. Reference tables

14.1 Datatype codes

CMB Value	Datatype
`00`	Untyped / bit-processing / not used
`01`	Binary
`10`	Integer
`11`	Decimal

The same encoding is used in CMB 1–2 (primary datatype) and, where applicable, in CMB 7–8 (immediate-value datatype or conversion destination type).

14.2 Condition codes

The 30 shared condition suffixes used by Jump, Call, and If commands:

#	Suffix	Condition	#	Suffix	Condition
1	Z	Zero	16	NNeg	Not negative
2	NZ	Not zero	17	Evn	Even
3	Blw	Below	18	Odd	Odd
4	BlwE	Below or equal	19	Ovr	Overflow
5	LT	Less than	20	NOvr	No overflow
6	LE	Less than or equal	21	Und	Underflow
7	EQ	Equal	22	NUnd	No underflow
8	NE	Not equal	23	Brw	Borrow
9	GE	Greater than or equal	24	NBrw	No borrow
10	GT	Greater than	25	Cry	Carry
11	AbvE	Above or equal	26	NCry	No carry
12	Abv	Above	27	THi	High-end truncation
13	Pos	Positive	28	NTHi	No high-end truncation
14	NPos	Not positive	29	TLo	Low-end truncation
15	Neg	Negative	30	NTLo	No low-end truncation

IfIRQ is specific to the If family and is not part of the shared set.

14.3 Family quick reference

Section	Family	ISA command(s)	Key CMB usage
4.2	Execution control	NOP, Brk, Retrn, ClrCF	All zero
4.3	Conditional block	If*, EndIf	All zero (condition is in opcode)
5	Jump	Jmp, JmpZ, …, JmpNTLo	1–2 imm type, 3 imm ind, 4 PC-rel, 5 backward, 6–8 CEB clear count
6	Call	Call, CallZ, …, CallNTLo	1–2 imm type, 3 imm ind, 4 PC-rel, 5 backward, 6–8 unused
7.2	Load / Store	Load, Stor	3 imm ind (count), 4 direction, 5 offset enable, 6 multiplier enable
7.3	Push / Pull	Push, Pull	3 imm ind, 7–8 imm type
7.4	Copy / Move	Copy, Move	1–2 datatype, 4 direction
7.5	Read / Write (I/O)	Read, Write	3 imm ind, 4 direction
7.6	Register utilities	ClrReg, Swap	All zero
7.7 / 13.1	SetVal / SetTxt	SetImm	3 imm ind (always `1`), 4 val/txt selector
8.3	Ordinary arithmetic	Add, Sub, Mul, Div	1–2 datatype, 3 imm ind, 4 sat, 5 carry/borrow, 6 set
8.4	Fused multiply-add	MulAdd	Same as 8.3; 5 unused
8.4	Divide with remainder	DivRem	Same as 8.3; 5 = `1` DivRem discriminator
8.5	Apply-to	AddTo, SubTo, MulTo, DivTo	Kiss-50: 1–2 datatype, 4 Sat, 5 Opr1 immediate value selector
8.6	Incr / Decr	Incr, Decr	1–2 datatype, 3 imm ind, 6 set, 7–8 imm type
9	Rotate and shift	Rot, Shift, SShift	1–2 datatype, 3 imm ind, 4 direction, 5 unused (=0), 6 set, 7–8 imm type
10.3	Ordinary logic	And, AndN, NAnd, Or, OrN, NOr, XOr, NXOr	1–2 datatype, 6 set
10.4	Not / Neg	Not, Neg	1–2 datatype, 6 set
10.6	BitFlip	BitFlip	1–2 = `00`, 6 set
10.7	Single-bit set/test	BitSet, BitTst	1–2 = `00`, 3 position imm, 4 on/off, 6 set
10.8	Bit-field	BitRead, BitWrite, BitClear	1–2 = `00`, 3 pos imm, 4 width imm, 6 set
10.9	Bit counting	BitCnt, BitCntTot	1–2 = `00`, 3 lead/trail, 4 on/off, 6 set
10.10	Bit finding	BitFind	1–2 = `00`, 3 first/last, 4 on, 6 set
10.11	Bit / byte reversal	BitRev, ByteRev	1–2 = `00`, 6 set
11	Test / Compare / Bounds	Test, Compare, ChkBnds	1–2 datatype, 6 required `1`
12	Conversion	Cvt	1–2 source type, 7–8 dest type

15. Examples

The following examples show common assembler patterns with brief explanations of what the assembler lowers each form into.

15.1 Typed arithmetic with flag update

```
I.AddSet R1WA R1WB R1WC
```

Add Integer R1WA + R1WB, write the result to R1WC, and update the arithmetic status flags. The assembler emits ISA command Add with CMB 1–2 = 10 (Integer), CMB-6 = 1 (Set), and all other CMB bits zero. Operand 1 = R1WA (sR1), Operand 2 = R1WB (sR2), Operand 5 = R1WC (dR5). Operands 3 and 4 are zero.

15.2 Conditional block

```
I.Compare R1WA R1WB
IfEQ
  I.Move R1WA R1WC
EndIf
```

Compare two Integer values; if they are equal, move R1WA to R1WC. I.Compare lowers to ISA Compare with CMB 1–2 = 10 and CMB-6 = 1; IfEQ lowers to opcode IfEQ with all CMB bits zero; I.Move lowers to ISA Move with CMB 1–2 = 10; EndIf closes the nested level. The I.Move inside the IfEQ block carries CEB = 11000 to mark it as belonging to the first nested level.

15.3 Loop with conditional jump

```
I.AddTo #I:10 R1WA R1WB R1WC
loop:
  I.DecrSet #I:1 R1WA
  BJmpNZ loop
```

Initialise by adding 10 to each of R1WA, R1WB, and R1WC. Then enter a decrement-and-branch loop: I.DecrSet subtracts 1 from R1WA and updates the status flags; BJmpNZ loop branches backward to loop while the zero flag is clear. BJmpNZ lowers to ISA JmpNZ with CMB-4 = 1 (PC-relative) and CMB-5 = 1 (backward); the assembler computes the offset and emits it as operand 1.

15.4 Typed immediate via SetVal

```
SetVal R1WA #I:-5
SetTxt R2WA "hi"
```

SetVal R1WA #I:-5 loads the signed Integer literal -5 into R1WA. The assembler encodes -5 in Kiss ones-complement (MSB = 0 because the value is negative), packs it into the 50-bit payload right-justified, places R1WA into operand 1, and sets CMB-3 = 1, CMB-4 = 0. SetTxt R2WA "hi" loads packed character codes for "hi" into R2WA with CMB-4 = 1 (text / left-aligned).

15.5 Load with offset and multiplier

```
LoadROfsMul R1WE R1WA R1WB R1WC R1WD
```

Load R1WE bytes from memory address R1WA + (R1WB × R1WC), right-aligned, into R1WD. The assembler emits ISA Load with CMB-4 = 0 (right-aligned), CMB-5 = 1 (offset enable), CMB-6 = 1 (multiplier enable); operands are byte count (iR1), base address (asR2), offset (sR3), multiplier

(sR4), destination (dR5).

15.6 Copy with sign-aware resize

```
I.CopyR R1WA R2WB R2WC
```

Copy Integer value from R1WA to both R2WB and R2WC, right-aligned. Because the type is Integer and the source and destination register widths differ (W-class vs 2W-class), the assembler emits ISA Copy with CMB 1–2 = 10 (Integer) and CMB-4 = 0 (right-aligned); the processor sign-extends the value on the fly, padding the extra high-order bits with NOT(sign bit). Each fanout destination is independently resized.

16. Instruction format reference by family

This appendix collects each family's CMB and operand pattern into a single compact table per family. The CEB field is omitted because it is always 00000 for an unconditional instance; conditional placement is governed by the surrounding If* / EndIf context or the Jump / Call CEB clear count.

Letter placeholders in the CMB column are bits that vary across the assembler variants within that family; their encoding is given by the family's CMB-usage section earlier in this document and summarised in the per-family legend below each table. A — in an operand column marks an unused slot (must be 0).

16.1 Execution control

Command	CMB	Opr 1	Opr 2	Opr 3	Opr 4	Opr 5	Description
NOP	`00000000`	—	—	—	—	—	No operation
Brk	`00000000`	—	—	—	—	—	Breakpoint
Retrn	`00000000`	—	—	—	—	—	Return; restore state from shadow call stack
ClrCF	`00000000`	—	—	—	—	—	Clear CEB state

16.2 Conditional block (If* / EndIf)

Command	CMB	Opr 1–5	Description
IfIRQ	`00000000`	—	Open If block if IRQ pending
IfZ	`00000000`	—	Open If block if Z set
IfGT	`00000000`	—	Open If block if signed greater-than
IfNCry	`00000000`	—	Open If block if Carry clear
EndIf	`00000000`	—	Close one active If level

33 If* variants plus EndIf in total — see the condition-codes table in §14.2 for the complete suffix set (which includes IfTHi/IfNTHi/IfTLo/IfNTLo for the truncation-direction flags added 2026-05-16). All variants share the same all-zero CMB and empty operand list.

16.3 Jump

Command	CMB	Opr 1	Opr 2–5	Description
Jmp	JJI00CCC	iR1	—	Unconditional absolute jump; PC = Opr1
FJmp	JJI10CCC	iR1	—	Unconditional forward-relative; PC = PC + Opr1
BJmp	JJI11CCC	iR1	—	Unconditional backward-relative; PC = PC − Opr1
JmpEQ	JJI00CCC	iR1	—	Absolute jump if equal
FJmpNZ	JJI10CCC	iR1	—	Forward-relative jump if non-zero
BJmpLE	JJI11CCC	iR1	—	Backward-relative jump if less-than-or-equal

Legend:

- JJ (CMB 1–2) = immediate-value data type (used when I = 1).
- I (CMB-3) = operand-1 form: 0 register, 1 immediate.
- CMB-4 / CMB-5 = direction selector (see §5.2).
- CCC (CMB 6–8) = CEB clear count.

31 base mnemonics × 3 direction forms = 93 distinct Jump assembler mnemonics. See §14.2 for the full condition suffix list.

16.4 Call

Command	CMB	Opr 1	Opr 2–5	Description
Call	JJI00000	iR1	—	Unconditional absolute call
FCall	JJI10000	iR1	—	Forward-relative call
BCall	JJI11000	iR1	—	Backward-relative call
CallEQ	JJI00000	iR1	—	Absolute call if equal
FCallNZ	JJI10000	iR1	—	Forward-relative call if non-zero
BCallLE	JJI11000	iR1	—	Backward-relative call if less-than-or-equal

Legend: same as Jump except CMB 6–8 = 0 0 0 (unused). 31 × 3 = 93 distinct Call assembler mnemonics. Calls save R1–R4, SF, CEB state, and special registers to the shadow call stack and enter the subroutine with CEB = 00000.

16.5 Load and Store

Command	CMB	Opr 1	Opr 2	Opr 3	Opr 4	Opr 5	Description
Load	00ID0000	iR1	asR 2	—	—	dR5	Load Opr1 bytes from mem[R2]
Load	00ID1000	iR1	asR 2	sR3	—	dR5	Load from mem[R2 + R3]
Load	00ID1100	iR1	asR	sR3	sR4	dR5	Load from mem[R2 + R3 × R4]

Command	CMB	Opr 1	Opr 2	Opr 3	Opr 4	Opr 5	Description
			2				
Stor	00ID0000	iR1	sR2	adR 3	—	—	Store Opr1 bytes to mem[R3]
Stor	00ID1000	iR1	sR2	adR 3	sR4	—	Store to mem[R3 + R4]
Stor	00ID1100	iR1	sR2	adR 3	sR4	sR5	Store to mem[R3 + R4 × R5]

Legend:

- I (CMB-3) = byte-count form: 0 register, 1 immediate.
- D (CMB-4) = direction: 0 right (*R), 1 left (*L).
- CMB-5 / CMB-6 select the address-computation mode (base, +offset, +offset×mul).

16.6 Push and Pull

Command	CMB	Opr 1	Opr 2	Opr 3	Opr 4	Opr 5	Description
Push	00I000JJ	iR1	sR2	osR 3	osR 4	osR 5	Push R2 (...R5) onto stack R1
Pull	00I000JJ	iR1	dR2	odR 3	odR 4	odR 5	Pull from stack R1 into R2 (...R5)

Legend: I (CMB-3) = stack selector form; JJ (CMB 7–8) = immediate data type.

16.7 Copy and Move

Command	CMB	Opr 1	Opr 2	Opr 3	Opr 4	Opr 5	Description
Copy	TT0D0000	sR1	odR 2	odR 3	odR 4	dR5	R5 ← R1 (+ fanout)
Move	TT0D0000	sR1	odR 2	odR 3	odR 4	dR5	R5 ← R1; R1 ← 0 after fanout

Legend: TT = data type; D = direction. Integer (TT = 10) is sign-aware resize.

16.8 Read and Write (I/O)

Command	CMB	Opr 1	Opr 2	Opr 3	Opr 4	Opr 5	Description
Read	00ID0000	iR1	sR2	—	—	dR5	R5 ← port[R2], Opr1 bytes
Write	00ID0000	iR1	sR2	sR3	—	—	port[R2] ← R3, Opr1 bytes

Legend: I = byte-count form; D = direction.

16.9 Register utilities

Command	CMB	Opr 1	Opr 2	Opr 3	Opr 4	Opr 5	Description
ClrReg	00000000	odR 1	odR 2	odR 3	odR 4	dR5	R5 ← 0 (+ fanout)
Swap	00000000	sdR 1	—	—	—	sdR 5	Exchange R1 ↔ R5

16.10 SetVal and SetTxt (SetImm)

Command	CMB	Opr 1	Opr 2	Opr 3	Opr 4	Opr 5	Description
SetVal (SetImm numeric)	00100000	dR1	imm	imm	imm	imm	R1 ← 50-bit numeric payload (right-aligned)
SetTxt (SetImm text)	00110000	dR1	imm	imm	imm	imm	R1 ← 50-bit text payload (left-aligned)

Special-form layout: destination is Opr 1, payload occupies Opr 2–5 right-justified (bits 41–50 are unused padding).

16.11 Ordinary arithmetic (Add, Sub, Mul, Div)

Command	CMB	Opr 1	Opr 2	Opr 3	Opr 4	Opr 5	Description
Add	TTIACS00	sR1	sR2	osR 3	osR 4	dR5	R5 = R1 + R2 (+R3 +R4)
Sub	TTIACS00	sR1	sR2	osR 3	osR 4	dR5	R5 = R1 − R2 (−R3 −R4)
Mul	TTIA0S00	sR1	sR2	osR 3	osR 4	dR5	R5 = R1 × R2 (×R3 ×R4)
Div	TTIA0S00	sR1	sR2	osR 3	osR 4	dR5	R5 = R1 ÷ R2 (÷R3 ÷R4)

Legend: TT data type; I imm ind; A saturation; C carry (Add) / borrow (Sub) — must be 0 for Mul/Div; S Set Status Flags.

16.12 Fused multiply-add (MulAdd)

Command	CMB	Opr 1	Opr 2	Opr 3	Opr 4	Opr 5	Description
MulAdd	TTIA0S00	sR1	sR2	sR3	—	dR5	R5 = (R1 × R2) + R3

CMB-5 unused (must be 0); no carry/borrow per 2026-03-28 decision. Fixed 4-operand form.

16.13 Divide with remainder (DivRem)

Command	CMB	Opr 1	Opr 2	Opr 3	Opr 4	Opr 5	Description
DivRem	TTIA1S00	sR1	sR2	—	dR4	dR5	R5 = R1 ÷ R2 (quotient); R4 = remainder

CMB-5 = 1 discriminates DivRem from plain Div. Saturation is valid; carry/borrow not valid.

16.14 Apply-to (AddTo, SubTo, MulTo, DivTo)

Command	Kiss-50 CMB	Opr 1	Opr 2	Opr 3	Description
AddTo	`TT0AI`	iR1	sdR2	-	R2 += value
SubTo	`TT0AI`	iR1	sdR2	-	R2 −= value
MulTo	`TT0AI`	iR1	sdR2	-	R2 ×= value
DivTo	`TT0AI`	iR1	sdR2	-	R2 ÷= value

Only the base form and Sat are valid. In compact Kiss-50, CMB-4 (A) is saturation and CMB-5 (I) selects whether operand 1 is a compact immediate value (1) or a value register (0). Per the 2026-03-28 decision, Set, Cry, and Brw are not valid for the Apply-to family.

16.15 Increment and Decrement (Incr, Decr)

Command	Kiss-50 CMB	Opr 1	Opr 2	Opr 3	Description
Incr	`TTS0I`	iR1	sdR2	-	R2 += amount
Decr	`TTS0I`	iR1	sdR2	-	R2 −= amount

Amount-driven. In compact Kiss-50, CMB-3 is SetFlags, CMB-4 must be 0, and CMB-5 (I) selects whether operand 1 is a compact immediate amount (1) or an amount register (0). Sat, Cry, and Brw are not valid.

16.16 Rotate and shift (Rot, Shift, SShift)

Command	CMB	Opr 1	Opr 2	Opr 3	Opr 4	Opr 5	Description
Rot	`TTID0SJJ`	iR1	osd R2	osd R3	osd R4	sdR 5	Rotate R5 (+ targets) by R1 bits / digits (Decimal: digit-level)
Shift	`TTID0SJJ`	iR1	osd R2	osd R3	osd R4	sdR 5	Logical shift (Decimal: digit-level, sign-matched zero fill)
SShift	`TTID0SJJ`	iR1	osd R2	osd R3	osd R4	sdR 5	Signed (arithmetic) shift — Integer (`TT = 10`) only

Legend: D direction (0 right, 1 left). CMB-5 is unused (must be 0) — through-carry has been retired across the family.

16.17 Ordinary logic

Command	CMB	Opr 1	Opr 2	Opr 3	Opr 4	Opr 5	Description
And	`TT000S00`	sR1	sR2	—	—	dR5	R5 = R1 ∧ R2
AndN	`TT000S00`	sR1	sR2	—	—	dR5	R5 = R1 ∧ ¬R2
NAnd	`TT000S00`	sR1	sR2	—	—	dR5	R5 = ¬(R1 ∧ R2)
Or	`TT000S00`	sR1	sR2	—	—	dR5	R5 = R1 ∨ R2
OrN	`TT000S00`	sR1	sR2	—	—	dR5	R5 = R1 ∨ ¬R2
NOr	`TT000S00`	sR1	sR2	—	—	dR5	R5 = ¬(R1 ∨ R2)
XOr	`TT000S00`	sR1	sR2	—	—	dR5	R5 = R1 ⊕ R2

Command	CMB	Opr 1	Opr 2	Opr 3	Opr 4	Opr 5	Description
NXOr	TT000S00	sR1	sR2	—	—	dR5	R5 = ¬(R1 ⊕ R2)

16.18 Not, Neg, BitFlip

Command	CMB	Opr 1	Opr 2	Opr 3	Opr 4	Opr 5	Description
Not	TT000S00	osd R1	osd R2	osd R3	osd R4	sdR 5	Invert R5 (+ targets) in place
Neg	TT000S00	—	—	—	—	sdR 5	R5 = −R5 (Integer or Decimal only)
BitFlip	00000S00	osd R1	osd R2	osd R3	osd R4	sdR 5	Invert R5 (+ targets) in place

Neg is invalid for Binary (TT = 01). With Set, Not is single-target only.

16.19 Single-bit set/test

Command	CMB	Opr 1	Opr 2–4	Opr 5	Description
BitSet (on)	00I10000	iR1	—	sdR5	Set bit at Opr1 in R5 to 1
BitSet (off)	00I00000	iR1	—	sdR5	Set bit at Opr1 in R5 to 0
BitTst (on)	00I10100	iR1	—	sR5	Z = 1 if bit Opr1 of R5 is 1
BitTst (off)	00I00100	iR1	—	sR5	Z = 1 if bit Opr1 of R5 is 0

Legend: I = position form (register or immediate). CMB-4 = On/Off selector.

16.20 Bit-field (BitRead, BitWrite, BitClear)

Command	CMB	Opr 1	Opr 2	Opr 3	Opr 4	Opr 5	Description
BitRead	00PW0S00	iR1	iR2	sR3	dR4	—	R4 = low-aligned field from R3
BitWrite	00PW0S00	iR1	iR2	sR3	sdR 4	—	Insert low Opr2 bits of R3 into R4 at Opr1
BitClear	00PW0S00	iR1	iR2	sdR 3	—	—	Clear Opr2 bits of R3 starting at Opr1

Legend: P = position form; W = width form.

16.21 Bit counting and finding

Command	CMB	Opr 1	Opr 2–4	Opr 5	Description
BitCntTot (on)	00010S00	sR1	—	dR5	R5 = popcount of 1 bits
BitCntTot (off)	00000S00	sR1	—	dR5	R5 = popcount of 0 bits
BitCnt (lead on)	00110S00	sR1	—	dR5	Count of leading 1 bits
BitCnt (lead off)	00100S00	sR1	—	dR5	Count of leading 0 bits
BitCnt (trail on)	00010S00	sR1	—	dR5	Count of trailing 1 bits
BitCnt (trail off)	00000S00	sR1	—	dR5	Count of trailing 0 bits

Command	CMB	Opr 1	Opr 2–4	Opr 5	Description
BitFind (first)	`00110S00`	sR1	—	dR5	First 1-bit from left (0 if none; Z set when none)
BitFind (last)	`00010S00`	sR1	—	dR5	Last 1-bit from right (0 if none; Z set when none)

See §10.9 / §10.10 for the exact assembler mnemonic → ISA command + CMB mapping.

16.22 Bit and byte reversal

Command	CMB	Opr 1	Opr 2–4	Opr 5	Description
BitRev	`00000S00`	sR1	—	dR5	R5 = R1 bit-reversed
ByteRev	`00000S00`	sR1	—	dR5	R5 = R1 byte-reversed (10-bit units)

16.23 Test, Compare, Bounds

Command	CMB	Opr 1	Opr 2	Opr 3	Opr 4	Opr 5	Description
Test	`TT000100`	sR1	—	—	—	—	Compare R1 against zero; set flags
Compare	`TT000100`	sR1	sR2	—	—	—	Compare R1 vs R2; set flags
ChkBnds	`TT000100`	sR1	sR2	sR3	—	—	Test R2 ≤ R1 ≤ R3; set flags

CMB-6 is required 1 for every instance; the family exists solely to set flags.

16.24 Conversion (Cvt)

Command	CMB	Opr 1	Opr 2–4	Opr 5	Description
Cvt	`SS0000DD`	sR1	—	dR5	R5 = R1 converted from source type SS to destination type DD

Assembler spells conversions with a double-dotted source-destination prefix: B.I.Cvt, I.D.Cvt, D.B.Cvt, etc.